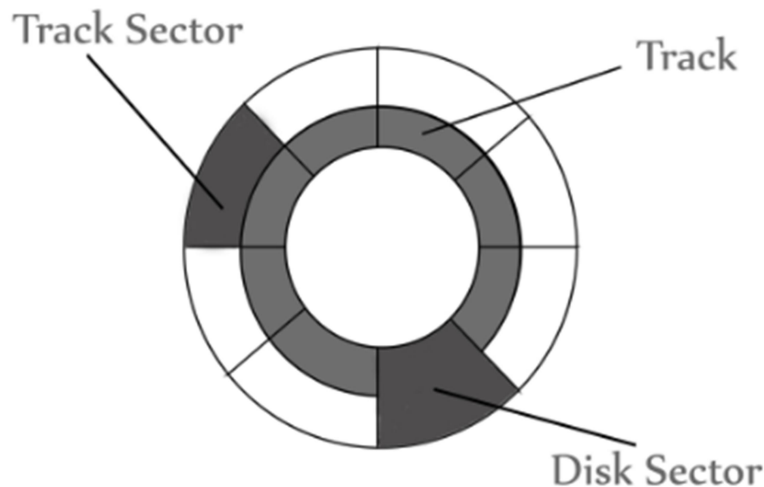


Unit 5: Disk Management

Disk Structure

- Disk is a secondary storage device that is utilized to store data.
- Disks give a way to store a lot of data to modern computer.
- Examples include: Hard Disk, Floppy Disk



Structure of Disk

- The disk is partitioned into tracks. Each track is additionally separated into parts.
- The point to be noted here is that external tracks are greater in size than the internal tracks however they contain a similar number of sectors and have equivalent storage limit.
- This is on the grounds that the storage thickness is high in sectors of the inward tracks where as the bits are meagrely arranged in sectors of the external tracks.
- Some space of each sector is utilized for formatting.
- In this way, the genuine limit of a segment is not exactly the given limit.
- Read-Write(R-W) head moves over the rotating hard disk. It is this Read-Write head that plays out all the read and write tasks on the disk and subsequently, position of the R-W head is a concern.
- To perform a read or write activity on a memory area, we have to put the R-W head over that position. Some significant terms must be noted here:
 1. **Seek time** – The time taken by the R-W head to arrive at the ideal track from it's present position.
 2. **Rotational latency** – Time taken by the sector to go under the R-W head.
 3. **Data transfer time** – Time taken to move the required measure of data. It relies on the rotational speed.
 4. **Controller time** – The processing time taken by the controller.
 5. **Average Access time** – seek time + Average Rotational latency + data transfer time + controller time .

- Note: Average Rotational dormancy is for the most part $1/2 \times (\text{Rotational latency})$.
- In questions, if the seek time and controller time isn't referenced, take them to be zero.
- On the off chance that the measure of data to be moved isn't given, accept that no data is being moved. Something else, ascertain the time taken to move the given measure of data.
- The average of rotational latency is taken when the present position of R-W head isn't given. Since, the R-W might be as of now present at the ideal position or it may take an entire present to get the ideal sector under the R-W head. However, on the off chance that the present position of the R-W head is given, at that point the rotational inactivity must be determined.

Example –

Consider a hard disk with:

2 surfaces

32 tracks/surface

64 sectors/track

128 bytes/sector

1. What is the limit of the hard disk?

Disk capacity = surfaces * tracks/surface * sectors/track * bytes/sector

Disk capacity = $2 * 32 * 64 * 128$

Disk capacity = 4 MB

2. The disk is rotating at 3600 RPM, what is the data move rate?

60 sec - > 3600 revolutions

1 sec - > 60 revolutions

Data transfer rate = number of rotations every second * track capacity * number of surfaces
(since 1 R-W head is utilized for each surface)

Data move rate = $60 * 64 * 128 * 2$

Data move rate = 5.5 MB/sec

3. The disk is turning at 3600 RPM, what is the normal access time?

Since, seek time, controller time and the amount of data to be moved isn't given, we consider all the three terms as 0.

Thusly, Average Access time = Average rotational delay

Rotational latency => 60 sec - > 3600 rotations

1 sec - > 60 rotations

Rotational inactivity = $(1/60)$ sec = 16.67 msec.

Average Rotational latency = $(16.67)/2 = 8.33$ msec.

Average Access time = 8.33 msec.

Disk Scheduling

- File systems must be accessed in a proficient way, particularly with hard drives, which are the slowest part of a computer.
- As a computer manages numerous processes over some undefined time frame, a list of requests to access to the disk develops.
- For productivity purposes, all solicitations (from all processes) are collected together.
- The procedure that the operating system uses to figure out which request to fulfil first is called disk scheduling.
- Disk scheduling is finished by operating systems to plan I/O requests arriving for the disk. Disk scheduling is otherwise called I/O planning.
- Disk scheduling is significant because:
- Various I/O requests may arrive by various processes and just a single I/O request can be served at once by the disk controller. In this manner other I/O requests need to hold up in the holding up queued and should be scheduled.
- At least two request might be a long way from one another so can bring about more greater disk arm development.
- Hard drives are one of the slowest parts of the computer system and along these lines should be gotten to in a proficient way.

Disk Scheduling Terms

- There are many Disk Scheduling Algorithms however before talking about them we should understand some of the significant terms:
- **Seek Time:** Seek time is the time taken to find the disk arm to a defined track where the data is to be read or write. So the disk scheduling algorithm that gives minimum average seek time is better.
- **Rotational Latency:** Rotational Latency is the time taken by the ideal sector of disk to pivot into a position with the goal that it can get to the read/compose heads. So the disk scheduling algorithm that gives least rotational latency is better.
- **Transfer Time:** Transfer time is the time to move the data. It relies upon the rotating rate of the disk and number of bytes to be moved.
- **Disk Access Time:** Disk Access Time is:
$$\text{Disk Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$
- **Disk Response Time:** Response Time is the normal of time spent by a request waiting to perform out its I/O activity. Average Response time is the reaction time of the all request. Variance Response Time is proportion of how individual request are adjusted as for normal reaction time. So the disk scheduling algorithm that gives least variance response time is better.

Disk Scheduling Algorithms

- The reason for disk scheduling is to decrease the total seek time.
- The different disks scheduling algorithms are given underneath. Every algorithm is conveying a few points of advantages and disadvantages.
- The impediment of every algorithm prompts the development of another algorithm.

1. FCFS (First Come First Serve) Scheduling Algorithm
2. SSTF (Shortest Seek Time First) Scheduling Algorithm
3. SCAN Scheduling Algorithm
4. C-SCAN Scheduling Algorithm
5. LOOK Scheduling Algorithm
6. C-LOOK Scheduling Algorithm

FCFS (First Come First Serve) Scheduling Algorithm

- FCFS is the least complex of all the Disk Scheduling Algorithms.
- In FCFS, the requests are accessed in the order in which they arrive in the disk queue.
- It benefits the I/O demands in the request in which they arrive. There is no starvation in this algorithm, each request is accessed.

Advantages:

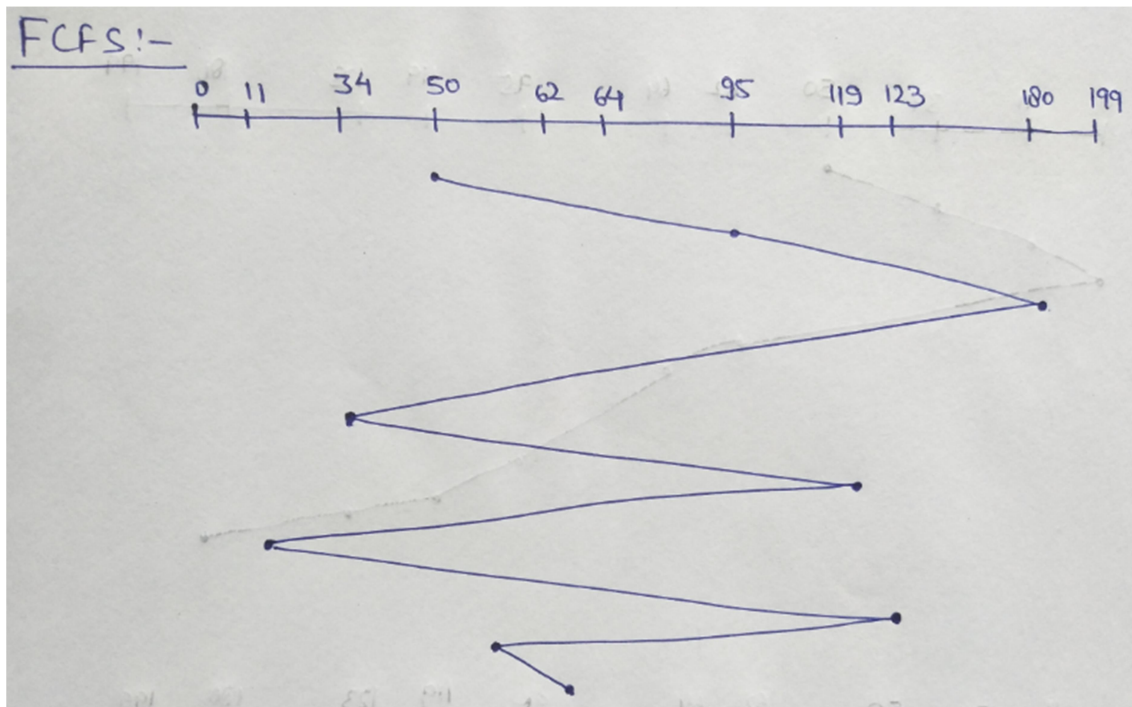
- Each request gets a reasonable chance
- No inconclusive deferment

Disadvantages:

- It does not takes any attempt not to upgrade seek time
- May not give the most ideal administration

Example:

Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.



$$\begin{aligned}
 \text{Total Seek Time} &= |50 - 95| + |95 - 180| + |180 - 34| + |34 - 119| + |119 - 11| + |11 - 123| + \\
 &\quad |123 - 62| + |62 - 64| \\
 &= 45 + 85 + 146 + 85 + 108 + 112 + 61 + 2 \\
 &= 644
 \end{aligned}$$

$$\text{Average Seek Time} = 644/8 = 80.5$$

SSTF (Shortest Seek Time First) Scheduling Algorithm

- In SSTF (Shortest Seek Time First), demands having most shortest seek time are executed first.
- In this way, the seek time of each request is determined ahead of time in the queue and after that they are scheduled by their calculated seek time.
- Accordingly, the request close to the disk arm will get executed first.
- SSTF is surely an improvement over FCFS as it diminishes the average response time and expands the throughput of system.

Advantages:

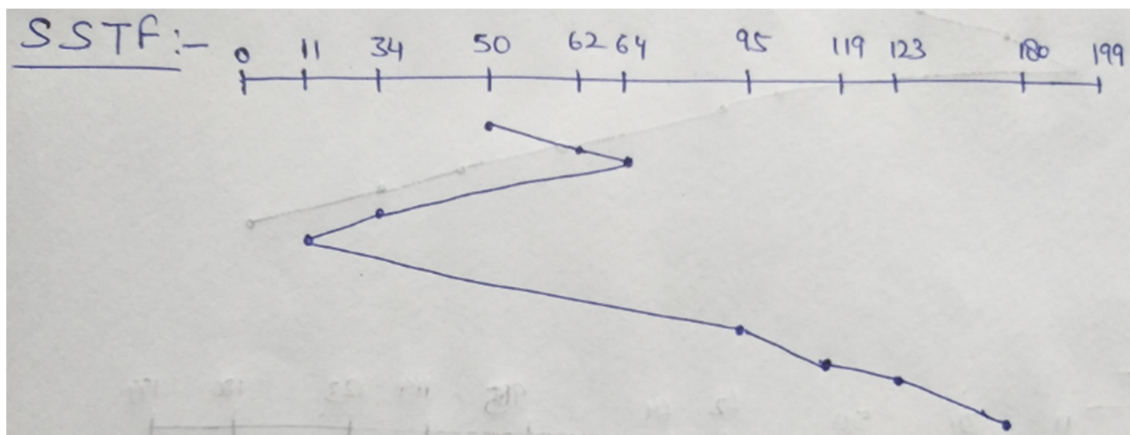
- Average Response Time diminishes
- Throughput increments

Disadvantages:

- Overhead to ascertain seek time ahead of time
- Can cause Starvation for a request on the off chance that it has higher seek time when contrasted with approaching requests.
- High fluctuation of response time as SSTF supports just a few requests.

Example:

Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.



$$\begin{aligned}
 \text{Total Seek Time} &= |50 - 62| + |62 - 64| + |64 - 34| + |34 - 11| + |11 - 95| + |95 - 119| + \\
 &\quad |119 - 123| + |123 - 180| \\
 &= 12 + 02 + 30 + 23 + 84 + 24 + 04 + 57
 \end{aligned}$$

$$= 236$$

$$\text{Average Seek Time} = 236/8 = 29.5$$

SCAN Scheduling Algorithm

- It is additionally called as Elevator Algorithm. In this algorithm, the disk arm moves into a specific heading till the end, fulfilling every one of the requests coming in its path, and then it turns back and moves in the opposite direction fulfilling requests coming in its way.
- It works in the manner a lift works, elevator moves toward a path totally till the last floor of that direction and afterward turns back.

Advantages:

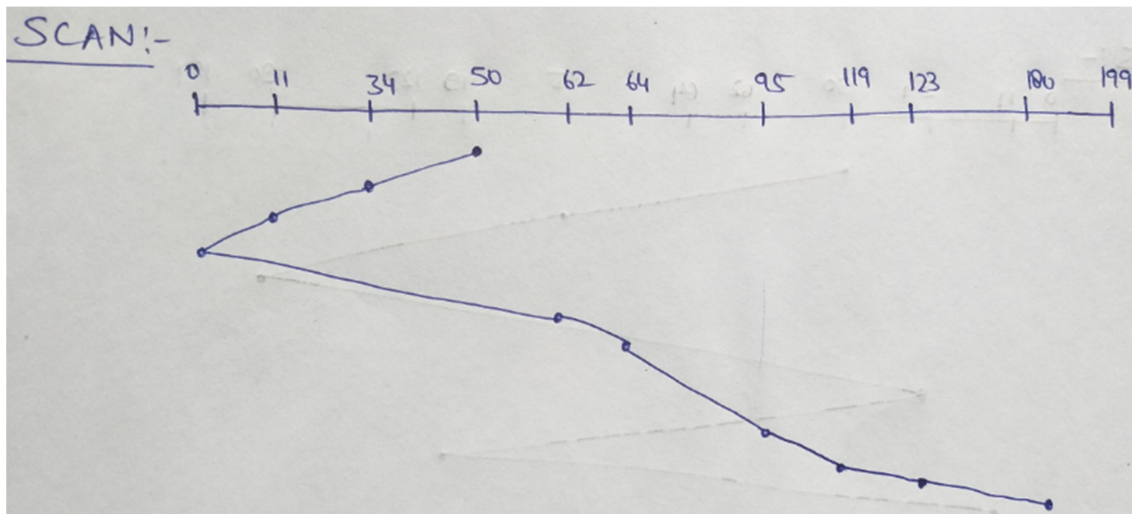
- High throughput
- Low difference in response time
- Average response time

Disadvantages:

- Long waiting time for requests for areas just visited by disk arm

Example:

Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.



$$\begin{aligned}
 \text{Total Seek Time} &= |50 - 34| + |34 - 11| + |11 - 0| + |0 - 62| + |62 - 64| + |64 - 95| + \\
 &\quad |95 - 119| + |119 - 123| + |123 - 180| \\
 &= 16 + 23 + 11 + 62 + 2 + 31 + 24 + 4 + 57 \\
 &= 230
 \end{aligned}$$

$$\text{Average Seek Time} = 230/8 = 28.75$$

C-SCAN Scheduling Algorithm

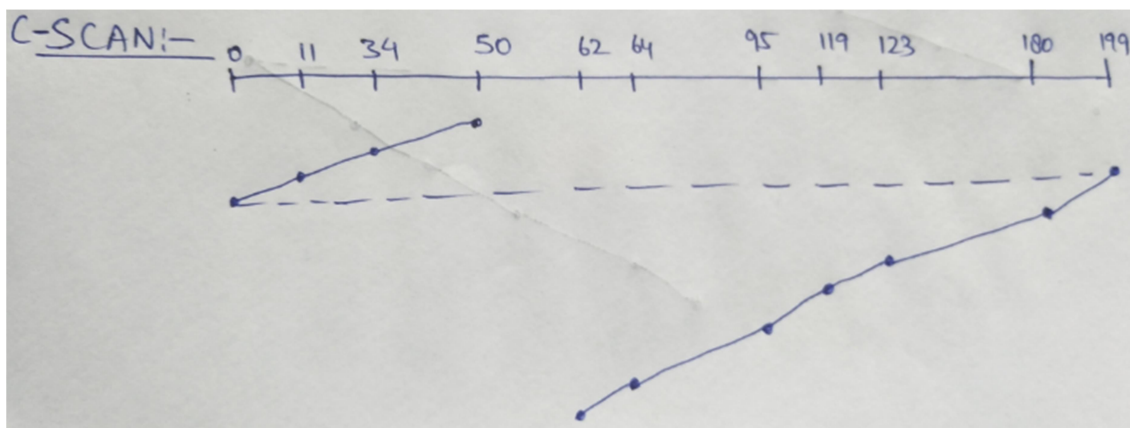
- In C-SCAN algorithm, the arm of the disk moves a specific way overhauling requests until it arrives at the last cylinder, at that point it bounces to the last chamber of the other way without adjusting any request then it turns back and start moving toward that path adjusting the rest of the requests.
- These circumstances are stayed away in CSCAN algorithm in which the disk arm as opposed to turning around its course goes to the opposite part of the disk and starts overhauling the requests from that point. Along these lines, the disk arm moves in a roundabout manner and this algorithm is additionally like SCAN algorithm and henceforth it is known as C-SCAN (Circular SCAN).

Advantages:

- Gives increasingly uniform wait time contrasted with SCAN

Example:

Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.



$$\begin{aligned}\text{Total Seek Time} &= |50 - 34| + |34 - 11| + |11 - 0| + |199 - 180| + |180 - 123| + |123 - 119| + \\ &\quad |119 - 95| + |95 - 64| + |64 - 62| \\ &= 16 + 23 + 11 + 19 + 57 + 4 + 24 + 31 + 2 \\ &= 187\end{aligned}$$

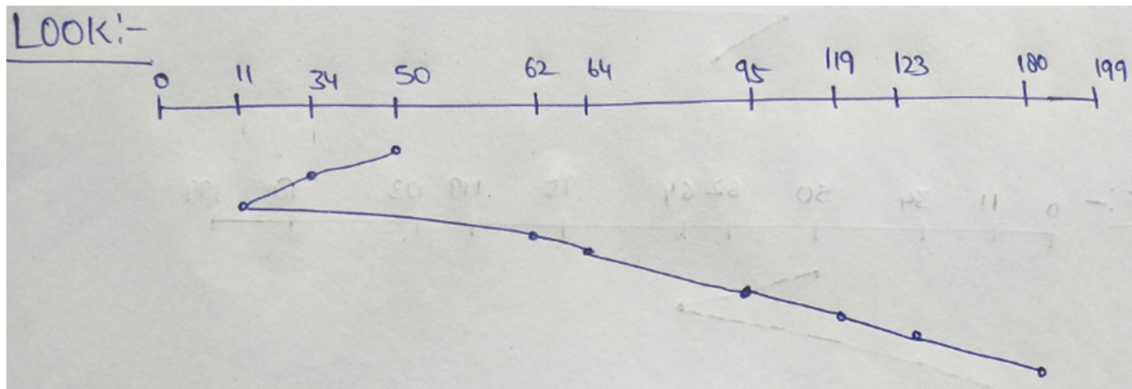
$$\text{Average Seek Time} = 187/8 = 23.375$$

LOOK Scheduling Algorithm

- It is like the SCAN disk scheduling algorithm with the exception of the distinction that the disk arm regardless of setting off to the end of the disk goes just to the last request to be adjusted before the head and after that turns around its direction from that point as it were.
- Subsequently it counteracts the additional postpone which happened because of superfluous traversal to the end of the disk.

Example:

Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.



$$\begin{aligned}
 \text{Total Seek Time} &= |50 - 34| + |34 - 11| + |11 - 62| + |62 - 64| + |64 - 95| + |95 - 119| + \\
 &\quad |119 - 123| + |123 - 180| \\
 &= 16 + 23 + 51 + 2 + 31 + 24 + 4 + 57 \\
 &= 208
 \end{aligned}$$

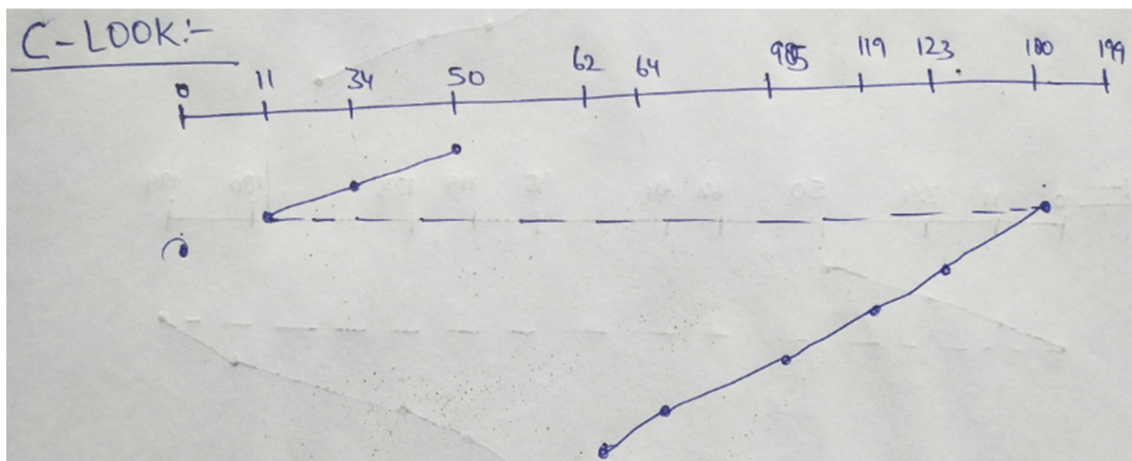
$$\text{Average Seek Time} = 208/8 = 26$$

C-LOOK Scheduling Algorithm

- As LOOK is like SCAN algorithm, in comparable way, C-LOOK is like C-SCAN disk scheduling algorithm.
- In C-LOOK, the disk arm despite setting off to the end goes just to the last request to be adjusted before the head and after that from that point goes to the opposite end's last request.
- Along these lines, it likewise forestalls the additional postpone which happened because of pointless traversal to the part of the disk.

Example:

Given the accompanying line - 95, 180, 34, 119, 11, 123, 62, 64 with the Read-write head at first at the track 50 and the tail track being at 199 given us now a chance to examine the various calculations.



$$\begin{aligned}\text{Total Seek Time} &= |50 - 34| + |34 - 11| + |180 - 123| + |123 - 119| + |119 - 95| + |95 - 64| + \\ &\quad |64 - 62| \\ &= 16 + 23 + 57 + 4 + 24 + 31 + 2 \\ &= 157\end{aligned}$$

$$\text{Average Seek Time} = 157/8 = 19.625$$